МИНОБРНАУКИ РОССИИ

Федеральное государственное бюджетное образовательное учреждение высшего образования

«МИРЭА — Российский технологический университет»

РТУ МИРЭА

Институт перспективных технологий и индустриального программирования (ИПТИП)

Кафедра индустриального программирования (ИП)

ОТЧЕТ ПО ПРАКТИЧЕСКОЙ РАБОТЕ

по дисциплине «Программирование корпоративных индустриальных систем»

Практическое задание № 5

Студент группы ЭФМО-02-25 Бондарь А.Р. _____
(подпись)

Доцент Макиевский С.Е. _____
(подпись)

Отчет предоставлен «____»_____ 2026г

Москва 2026

СОДЕРЖАНИЕ

Введение 4
1 Клиент-серверная подсистема 5
1.1 Границы клиент-серверного контура 5
1.2 Проектирование API и контрактов 5
1.2.1 Единый формат ответа 5
1.2.2 Единый формат ошибки 5
1.2.3 Коды ошибок 6
1.2.4 Описание endpoint-ов 6
1.3 Спецификация OpenAPI (фрагмент) 7
1.4 Проектирование модели данных и ограничений 8
1.5 Реализация серверной части 10
1.6 Реализация клиента (пример консольного сценария) 11
1.7 Безопасность взаимодействия 11
1.8 Наблюдаемость и диагностика 12
1.9 Тестирование API 12
2 Транзакции и конкурентность 13
2.1 Выбор транзакционно критичных операций 13
2.2 Границы транзакции 13
2.3 Выбор уровней изоляции 14
2.4 Реализация конкурентного контроля 14
2.5 Параллельные сценарии и тесты конкурентности 14
2.6 Использование расширенного SQL 15
2.7 Диагностика блокировок 15
2.8 Метрики и критерии 16
2.9 Рекомендации по эксплуатации 16
3 Администрирование СУБД 17
3.1 Развёртывание PostgreSQL 17

3.2 Сетевой доступ и безопасность 17
3.3 Роли и права доступа 17
3.4 Журналирование и аудит 18
3.5 Резервное копирование 18
3.6 Проверка восстановления 18
3.7 Мониторинг и показатели 18
3.8 Оптимизация типовых запросов 19
3.9 Регламент обслуживания 19
4 Сопровождение и завершение проекта 20
4.1 Выбор стандартов и протоколов 20
4.2 Регламент версионирования API 20
4.3 Правила модификации схемы базы данных 20
4.4 Требования к наблюдаемости 21
4.5 Политика безопасности сопровождения 21
4.6 Документация 21
4.7 Тестирование и критерии качества релиза 21
4.8 CI/CD 22
4.9 Управление инцидентами 22
Заключение 23

ВВЕДЕНИЕ

Данный документ завершает серию практических работ по проектированию и реализации системы автоматизации библиотеки. На основе ранее разработанных моделей (пр.5–12) теперь проектируются и описываются:

- клиент-серверное взаимодействие (API, контракты, база данных);
- управление транзакциями и конкурентностью;
- административные процедуры для СУБД (PostgreSQL);
- регламенты сопровождения, версионирования, CI/CD и безопасности.

Все артефакты взаимосвязаны и готовы к реализации и эксплуатации.

1 КЛИЕНТ-СЕРВЕРНАЯ ПОДСИСТЕМА

1.1 Границы клиент-серверного контура

Выбраны следующие операции (не менее шести), реализующие ключевые сценарии:

Таблица 1 – Операции и права доступа

Операция	**Роль**	**Права доступа**
POST /api/readers	Библиотекарь	Создание читателя
GET /api/readers/{id}	Библиотекарь, Читатель	Просмотр данных (своих или всех)
POST /api/loans	Библиотекарь	Выдача книги
PUT /api/loans/{id}/return	Библиотекарь, Читатель	Возврат книги
POST /api/fines/{id}/pay	Читатель	Оплата штрафа
GET /api/books?search=...	Все (в т.ч. аноним)	Поиск книг в каталоге

Для работы были приняты некоторые допущения:

– На первом этапе интеграция с внешней платёжной системой эмулируется.

– Аутентификация через JWT, роли хранятся в токене.

1.2 Проектирование API и контрактов

1.2.1 Единый формат ответа

```
{
  "success": true,
  "data": { ... },
  "error": null
}
```

1.2.2 Единый формат ошибки

```
{
  "success": false,
  "data": null,
  "error": {
    "code": "ERROR_CODE",
    "message": "Понятное пользователю сообщение",
    "details": { ... }
  }
}
```

1.2.3 Коды ошибок

1. `READER_NOT_FOUND` — читатель не найден.
2. `BOOK_COPY_NOT_AVAILABLE` — экземпляр недоступен.
3. `LOAN_LIMIT_EXCEEDED` — превышен лимит выдач.
4. `FINE_ALREADY_PAID` — штраф уже оплачен.
5. `VALIDATION_FAILED` — ошибка валидации.
6. `ACCESS_DENIED` — недостаточно прав.

1.2.4 Описание endpoint-ов

Регистрация читателя

```
POST /api/readers
Request:
{
  "fullName": "Иванов Иван",
  "email": "ivanov@mail.ru",
  "passportData": "1234 567890",
  "phone": "+71234567890"
}
Response 201:
{
  "id": 1,
  "libraryCardNumber": "LIB-00001",
  "fullName": "...",
  "email": "...",
  "status": "Active"
}
```

Выдача книги

```
POST /api/loans
Request:
{
  "readerId": 1,
  "copyId": 101,
  "dueDate": "2026-04-01"
}
Response 201:
{
  "id": 1001,
  "readerId": 1,
  "copyId": 101,
  "loanDate": "2026-03-17",
  "dueDate": "2026-04-01",
  "status": "Active"
}
```

Возврат книги

```
PUT /api/loans/{id}/return
Request body:
{
    "returnDate": "2026-03-20"
} // опционально, по умолчанию now
Response 200:
{
  "id": 1001,
  "returnDate": "2026-03-20",
  "fineAmount": 50.0,
  "fineId": 5001
}
```

Оплата штрафа

```
POST /api/fines/{id}/pay
Response 200:
{
  "id": 5001,
  "status": "Paid",
  "paymentDate": "2026-03-20"
}
```

Поиск книг

```
GET /api/books?titleвойна=&authorтолстой=&genreроман=
Response 200:
{
  "items": [
    {
      "id": 10,
      "title": "Война и мир",
      "author": "Лев Толстой",
      "availableCopies": 3
    }
  ],
  "total": 1
}
```

1.3 Спецификация OpenAPI (фрагмент)

```
openapi: 3.0.0
info:
  title: Library API
  version: 1.0.0
paths:
  /api/readers:
    post:
      summary: Register a new reader
      requestBody:
```

```yaml
        required: true
        content:
          application/json:
            schema:
              \$ref: '#/components/schemas/RegisterReaderRequest'
      responses:
        '201':
          description: Reader created
          content:
            application/json:
              schema:
                \$ref: '#/components/schemas/ReaderResponse'
        '400':
          description: Validation error
          content:
            application/json:
              schema:
                \$ref: '#/components/schemas/ErrorResponse'
components:
  schemas:
    RegisterReaderRequest:
      type: object
      properties:
        fullName: { type: string }
        email: { type: string, format: email }
        passportData: { type: string }
        phone: { type: string }
      required: [fullName, email, passportData]
    ReaderResponse:
      type: object
      properties:
        id: { type: integer }
        libraryCardNumber: { type: string }
        fullName: { type: string }
        email: { type: string }
        status: { type: string, enum: [Active, Blocked, Deleted] }
    ErrorResponse:
      type: object
      properties:
        success: { type: boolean, default: false }
        error:
          type: object
          properties:
            code: { type: string }
            message: { type: string }
```

Для операций создания (`POST /readers`, `POST /loans`) рекомендуется использовать заголовок `Idempotency-Key`. При повторном запросе с тем же ключом возвращается ранее созданный ресурс.

1.4 Проектирование модели данных и ограничений

Таблица 2 – Таблица `Readers`

Колонка	**Тип**	**Ограничения**
id	bigserial	PRIMARY KEY
library_card_number	varchar(20)	UNIQUE NOT NULL
full_name	varchar(200)	NOT NULL
email	varchar(100)	UNIQUE NOT NULL
passport_data	varchar(50)	UNIQUE NOT NULL
phone	varchar(20)	
status	varchar(20)	NOT NULL DEFAULT ’Active’
created_at	timestamptz	DEFAULT now()
updated_at	timestamptz	
xmin	xid	

Таблица 3 – Таблица `Books`

Колонка	**Тип**	**Ограничения**
id	bigserial	PRIMARY KEY
isbn	varchar(20)	UNIQUE NOT NULL
title	varchar(500)	NOT NULL
publisher	varchar(200)	
publication_year	int	
pages	int	
description	text	

Таблица 4 – Таблица `Authors`

Колонка	**Тип**	**Ограничения**
id	bigserial	PRIMARY KEY
name	varchar(200)	UNIQUE NOT NULL

Таблица 5 – Таблица `BookAuthors`

Колонка	**Тип**	**Ограничения**
book_id	bigint	REFERENCES Books(id) ON DELETE CASCADE
author_id	bigint	REFERENCES Authors(id) ON DELETE CASCADE

Таблица 6 – Таблица `BookCopies`

Колонка	**Тип**	**Ограничения**
id	bigserial	PRIMARY KEY
book_id	bigint	REFERENCES Books(id)
inventory_number	varchar(50)	UNIQUE NOT NULL
status	varchar(20)	NOT NULL DEFAULT ’Available’
price	decimal(10,2)	
shelf_location	varchar(50)	
version	int	NOT NULL DEFAULT 0

Таблица 7 – Таблица `Loans`

Колонка	**Тип**	**Ограничения**
id	bigserial	PRIMARY KEY
reader_id	bigint	REFERENCES Readers(id)

Продолжение Таблицы 7

Колонка	**Тип**	**Ограничения**
copy_id	bigint	REFERENCES BookCopies(id)
loan_date	date	NOT NULL
due_date	date	NOT NULL
return_date	date	
status	varchar(20)	NOT NULL DEFAULT ’Active’
version	int	NOT NULL DEFAULT 0
created_at	timestamptz	DEFAULT now()
updated_at	timestamptz	

Таблица 8 – Таблица `Fines`

Колонка	**Тип**	**Ограничения**
id	bigserial	PRIMARY KEY
loan_id	bigint	UNIQUE REFERENCES Loans(id)
amount	decimal(10,2)	NOT NULL CHECK (amount > 0)
accrual_date	date	NOT NULL
payment_date	date	
status	varchar(20)	NOT NULL DEFAULT ’Unpaid’
version	int	NOT NULL DEFAULT 0

Индексы:

– `idx_loans_reader_status` на `(reader_id, status)` с условием `status IN ('Active', 'Overdue')`

– `idx_loans_copy_id` на `copy_id`

– `idx_fines_loan_id` на `loan_id`

– полнотекстовый поиск по `books` (GIN индекс на `title` и `description`)

1.5 Реализация серверной части

Используется ASP.NET Core 8, Entity Framework Core (Npgsql). Структура решения (как в пр.11):

– **Library.Domain** — модели, интерфейсы репозиториев;

– **Library.Application** — сервисы, команды, DTO;

– **Library.Infrastructure** — реализации репозиториев, EF Core DbContext, миграции;

– **Library.API** — контроллеры, middleware.

В приложении реализована централизованная обработка ошибок через глобальный middleware, который перехватывает все исключения и возвращает

клиенту стандартизированный ответ ErrorResponse, а для логирования используется ILogger с поддержкой корреляционного идентификатора — если клиент не передал его в заголовке X-Correlation-ID, система генерирует его автоматически, что позволяет отслеживать путь запроса. Валидация входных данных выполняется с помощью FluentValidation, обеспечивая чистоту команд, а конфигурация приложения хранится в appsettings.json, при этом чувствительные параметры (например, строки подключения или настройки JWT) можно гибко переопределять через переменные окружения.

1.6 Реализация клиента (пример консольного сценария)

Консольный клиент на C# (HttpClient) демонстрирует сценарий:

1. Регистрация читателя.
2. Поиск книги.
3. Выдача книги.
4. Возврат с просрочкой.
5. Оплата штрафа.

Код клиента не приводится полностью, но описывается логика и обработка ошибок (повторы при временных сбоях, проверка кодов ответа).

1.7 Безопасность взаимодействия

Аутентификация: JWT, выдаваемый после логина (в данном контуре логин не реализован, предполагается, что токен получен заранее). Роли (Librarian, Reader) в claims.

Авторизация: в контроллерах атрибуты `[Authorize(Roles = "Librarian")]`.

TLS: обязательно в production, в dev можно отключить.

Аудит: все изменяющие операции логируются с указанием пользователя (из JWT), времени, IP.

1.8 Наблюдаемость и диагностика

Логи: структурированные (JSON), обязательные поля: timestamp, level, correlationId, userId, path, method, statusCode, durationMs.

Метрики: Prometheus + Grafana (число запросов, latency, ошибки по кодам).

Трассировка: OpenTelemetry (Jaeger) для сквозной трассировки.

1.9 Тестирование API

Таблица 9 – Тест-кейсы API

ID	Описание	Ожидаемый результат
API-01	Создание читателя с валидными данными	201, читатель создан
API-02	Создание читателя с существующим email	409, ошибка
API-03	Выдача книги при доступном экземпляре	201, статус Loaned
API-04	Выдача книги при недоступном экземпляре	409, ошибка
API-05	Выдача книги читателю с блокировкой	403/409, ошибка
API-06	Возврат книги без просрочки	200, нет штрафа
API-07	Возврат книги с просрочкой	200, есть штраф
API-08	Оплата штрафа	200, статус Paid
API-09	Оплата уже оплаченного штрафа	409, ошибка
API-10	Поиск книг по названию	200, список
API-11	Доступ к чужому профилю читателем	403
API-12	Доступ библиотекаря к любому профилю	200

2 ТРАНЗАКЦИИ И КОНКУРЕНТНОСТЬ

2.1 Выбор транзакционно критичных операций

1. **Выдача книги** — меняет статус экземпляра, создаёт запись Loan. Должно быть атомарно.

2. **Возврат книги** — обновляет Loan, меняет статус экземпляра, создаёт Fine при просрочке.

3. **Оплата штрафа** — меняет статус Fine и проставляет paymentDate.

4. **Блокировка читателя** — меняет статус Reader, требует проверки отсутствия активных выдач? (в нашей модели блокировка разрешена, но выдача новых невозможна).

2.2 Границы транзакции

В коде транзакция охватывает все обращения к БД в рамках одного сервисного метода. Используется `UnitOfWork` (или `DbContext.SaveChanges()` в EF Core).

Пример для выдачи показан в листинге ниже.

```
public async Task<LoanDto> IssueBook(IssueBookCommand command) {
    using var transaction = await _context.Database.BeginTransactionAsync();
    try {
        var reader = await _readerRepo.GetByIdAsync(command.ReaderId);
        var copy = await _copyRepo.GetByIdAsync(command.CopyId);
        // бизнеспроверки-
        var loan = new Loan(reader, copy, DateTime.UtcNow, command.DueDate);
        await _loanRepo.AddAsync(loan);
        copy.Loan(); // меняет статус
        await _copyRepo.UpdateAsync(copy);
        await _unitOfWork.SaveChangesAsync();
        await transaction.CommitAsync();
        return _mapper.Map<LoanDto>(loan);
    } catch {
        await transaction.RollbackAsync();
        throw;
    }
}
```

Внешние вызовы, например, отправка email, выполняются после коммита, чтобы не включать их в транзакцию.

Компенсация не требуется, так как транзакция атомарна. В случае сбоя после коммита компенсируется повторной попыткой или логированием.

2.3 Выбор уровней изоляции

Таблица 10 – Уровни изоляции для операций

Операция	**Уровень изоляции**	**Обоснование**
Выдача книги	Repeatable Read (с оптимистичной блокировкой)	Уникальный индекс защищает от дублирования, но для дополнительной надёжности используем версию
Возврат книги	Read Committed	Достаточно, так как изменения затрагивают одну запись Loan и одну BookCopy
Оплата штрафа	Read Committed	Простое обновление одной записи

2.4 Реализация конкурентного контроля

Оптимистическая блокировка через поле `Version` (в PostgreSQL можно использовать `xmin` или отдельное целое). В EF Core настраивается как `IsRowVersion()`.

При конфликте (`DbUpdateConcurrencyException`) пользователю возвращается ошибка с предложением повторить операцию.

Пример обработки:

```
catch (DbUpdateConcurrencyException ex) {
    throw new ConcurrencyException(
        "Данные были изменены другим пользователем. Повторите операцию.");
}
```

2.5 Параллельные сценарии и тесты конкурентности

Тесты запускают несколько параллельных запросов (например, с `Parallel.ForEach`).

Сценарии:

1. Две параллельные выдачи одного экземпляра — только одна успешна, вторая получает ошибку.

2. Два параллельных возврата одной выдачи — первый успех, второй ошибка.

3. Оплата одного штрафа дважды — первый успех, второй ошибка.

4. Блокировка читателя и одновременная выдача ему книги — выдача отклоняется (проверка статуса читателя внутри транзакции).

5. Изменение статуса экземпляра (списание) и выдача — конфликт версий.

2.6 Использование расширенного SQL

Пример upsert для штрафа:

```
INSERT INTO fines (loan_id, amount, accrual_date, status, version)
VALUES (@loan_id, @amount, @accrual_date, 'Unpaid', 0)
ON CONFLICT (loan_id) DO NOTHING;
```

Пример обновления с проверкой версии:

```
UPDATE book_copies
SET status = @new_status, version = version + 1
WHERE id = @id AND version = @old_version;
```

Пример CTE для атомарного возврата:

```
WITH updated_loan AS (
    UPDATE loans
    SET return_date = @return_date, status = 'Returned', version = version + 1
    WHERE id = @id AND version = @version
    RETURNING *
)
INSERT INTO fines (loan_id, amount, accrual_date, status)
SELECT id, @amount, @return_date, 'Unpaid'
FROM updated_loan
WHERE @amount > 0;
```

2.7 Диагностика блокировок

1. Включить логирование медленных запросов (>500 мс).

2. Мониторить `pg_locks` для выявления долгих блокировок.

3. Настроить `deadlock_timeout` и логировать взаимоблокировки.

4. Метрики: количество deadlock’ов, среднее время ожидания блокировок.

2.8 Метрики и критерии

Доля конфликтов версий (число ConcurrencyException / общее число операций). Порог > 1% требуется анализ.

Среднее время транзакции.

Число повторных попыток (retries) на уровне приложения.

2.9 Рекомендации по эксплуатации

1. Пул соединений: размер = CPU*2+дисковые операции.
2. Таймауты: CommandTimeout = 30 с, TransactionTimeout = 15 с.
3. Индексы: по внешним ключам, по полям версий.
4. Регулярный analyze для обновления статистики.

3 АДМИНИСТРИРОВАНИЕ СУБД

3.1 Развёртывание PostgreSQL

Используется PostgreSQL 16 в Docker:

```
docker run -d --name library-db -e POSTGRES_PASSWORD=secret -p 5432:5432 postgres:16
```

Параметры конфигурации: `shared_buffers = 256MB`, `work_mem = 16MB`, `maintenance_work_mem = 64MB`. Директории для данных и WAL отдельно.

3.2 Сетевой доступ и безопасность

Доступ к базе данных ограничивается на уровне `pg_hba.conf` — разрешены подключения только с доверенных IP-адресов. Весь трафик шифруется с помощью SSL-сертификатов, что исключает перехват данных. Основные угрозы (несанкционированный доступ, SQL-инъекции) нейтрализуются настройками сети и экранированием запросов на стороне приложения.

3.3 Роли и права доступа

В PostgreSQL создаются четыре роли с минимально необходимыми привилегиями:

1. `db_admin` — полный доступ (для миграций и обслуживания).
2. `app_user` — права на чтение/запись таблиц (SELECT, INSERT, UPDATE, DELETE) для схемы `public`.
3. `readonly_user` — только SELECT для отчётов.
4. `auditor` — доступ к логам и журналам.

Пример создания роли приложения:

```
CREATE ROLE app_user LOGIN PASSWORD 'app_pass';
GRANT CONNECT ON DATABASE library TO app_user;
GRANT USAGE ON SCHEMA public TO app_user;
GRANT SELECT, INSERT, UPDATE, DELETE ON ALL TABLES IN SCHEMA public TO app_user;
```

3.4 Журналирование и аудит

Логирование настроено на запись ошибок и подключений в CSV-формате через включённый `logging_collector`. Медленные запросы (дольше 1 секунды) фиксируются с помощью параметра `log_min_duration_statement = 1000`. Для аудита изменений данных используются триггеры или расширение `pgaudit`.

3.5 Резервное копирование

Для реализации комбинированной стратегии каждую ночь выполняется полный дамп базы (`pg_dump` в пользовательском формате или `pg_basebackup`), а также ведётся непрерывное WAL-архивирование, позволяющее восстановить состояние на любой момент времени. Все резервные копии шифруются (например, GPG) и хранятся 30 дней в удалённом дата-центре или S3-хранилище.

Пример команды создания дампа:

```
pg_dump -U backup_user -Fc -f /backup/library_\$(date +\%Y\%m\%d).dump library
```

3.6 Проверка восстановления

Разработан и ежемесячно тестируется протокол восстановления. Приложение останавливается, создаётся новая чистая база, в неё загружается последний дамп (`pg_restore`), после чего проверяется целостность данных (количество записей, выполнение ключевых запросов) и приложение переключается на восстановленную БД. Целевое время восстановления — не более 15 минут.

3.7 Мониторинг и показатели

Для наблюдения за состоянием БД используется связка Prometheus и `postgres_exporter`. Собираются метрики: количество активных соединений,

загрузка CPU и диска, размер базы и индексов, частота deadlock-ов, длительность запросов.

Настроены алерты:

- Соединения > 90% от `max_connections`.
- Свободное место на диске < 10%.
- Deadlocks > 0 за 5 мин.

3.8 Оптимизация типовых запросов

Три наиболее частых запроса — поиск книг по названию и автору, получение активных выдач читателя и отчёт о задолжниках — были проанализированы через `EXPLAIN ANALYZE`. Для ускорения созданы индексы: GIN для полнотекстового поиска по текстовым полям, составной индекс по `loans(reader_id, status)` с условием `WHERE status IN ('Active', 'Overdue')`.

Пример индекса:

```
CREATE INDEX idx_loans_reader_status ON loans(reader_id, status)
    WHERE status IN ('Active', 'Overdue');
```

После создания запрос ускорился с 500 мс до 20 мс.

3.9 Регламент обслуживания

Ежедневно по расписанию (cron) выполняется `VACUUM ANALYZE` для обновления статистики и очистки мёртвых кортежей. Раз в месяц для таблиц с высокой интенсивностью обновлений (например, `loans`) проводится `REINDEX`. После массовых изменений данных статистика обновляется принудительно. Дополнительно ведётся мониторинг фрагментации индексов для своевременного принятия мер.

4 СОПРОВОЖДЕНИЕ И ЗАВЕРШЕНИЕ ПРОЕКТА

4.1 Выбор стандартов и протоколов

В проекте используются стандарты и протоколы: HTTP/1.1 и HTTP/2 с TLS 1.3, JWT (RFC 7519) для аутентификации, OpenAPI 3.0 для спецификации, REST как архитектурный стиль, а также PostgreSQL native protocol с SSL для работы с базой данных.

4.2 Регламент версионирования API

Версионирование API строится на следующих принципах: ломающие изменения (удаление полей, изменение типов или семантики) требуют новой версии, тогда как добавление полей или новых endpoint'ов считается не ломающим и может выполняться в текущей версии. Каждая выпущенная версия поддерживается не менее одного года после выхода следующей. Клиент указывает версию в URL, а ответ обязательно содержит заголовок `X-API-Version`.

4.3 Правила модификации схемы базы данных

Миграции базы данных выполняются через EF Core или FluentMigrator, каждая миграция сохраняется в отдельном файле в репозитории. Применение миграций в production происходит с минимальным временем простоя: сначала миграция на реплике, затем переключение трафика. Для отката создаётся обратная миграция (скрипт отката). При проектировании миграций следует избегать длительных блокировок таблиц – например, использовать `CONCURRENTLY` для индексов и добавлять столбцы с NULL.

4.4 Требования к наблюдаемости

Все логи должны быть в формате JSON с полями: `@timestamp`, level, correlationId, service, userId, path, method, statusCode, duration. Метрики включают QPS, латентность (p95, p99), частоту ошибок и бизнес-показатели (например, количество выдач). Сквозная трассировка обеспечивается передачей correlationId во все внутренние вызовы. Алерты срабатывают при превышении error rate 5% за 5 минут или p95 > 2 секунд.

4.5 Политика безопасности сопровождения

Секреты хранятся в Azure Key Vault, HashiCorp Vault или переменных окружения. Ключи JWT ротируются каждые 30 дней. Зависимости регулярно обновляются с помощью Dependabot. Права администраторов ограничены, а все действия логируются.

4.6 Документация

Репозиторий содержит README с инструкциями по запуску и настройке, OpenAPI-спецификацию в формате YAML, доступную по `/swagger`, архитектурные диаграммы в папке `/docs`, а также отдельные инструкции по деплою и эксплуатации.

4.7 Тестирование и критерии качества релиза

Обязательны модульные тесты (Domain, Application) с покрытием 100% ключевой логики, интеграционные тесты API, покрывающие все сценарии, нагрузочные тесты с требованием p95 < 500 мс при 100 RPS, а также контрактные тесты для проверки совместимости с клиентами. Релиз запрещается, если упали критические тесты, увеличилась доля ошибок на стенде staging или обнаружены уязвимости высокого уровня.

4.8 CI/CD

Непрерывная интеграция и доставка реализованы на базе GitLab CI или GitHub Actions. Пайплайн включает этапы: сборка, модульные тесты, интеграционные тесты, сканирование безопасности, деплой на staging, приёмочные тесты и выкат в production по стратегии blue-green. Артефактами являются Docker-образы и скрипты миграций. Откат выполняется переключением на предыдущий образ.

4.9 Управление инцидентами

Инциденты классифицируются по уровням: критичный (система недоступна), высокий, средний, низкий. Время реакции на критичный инцидент – 15 минут, на высокий – 1 час. Расследование ведётся с использованием correlationId для сбора логов, метрик и дашбордов. Постмортем обязателен для критичных и высоких инцидентов с описанием причин и планом предотвращения.

ЗАКЛЮЧЕНИЕ

Выполнены все практические работы в рамках сквозного проекта автоматизации библиотеки. Разработаны:

- модели требований и статическая структура (UML),
- модели взаимодействий (кооперации),
- модели состояний и деятельности,
- модульная структура библиотек и реализация фрагмента на C#,
- клиент-серверная подсистема (API, БД),
- транзакции и конкурентность,
- администрирование СУБД,
- регламенты сопровождения.

Полученный комплект может служить основой для реальной разработки и внедрения системы.